2nd Project Report

KITTI LiDAR and Camera classical fusion techniques comparison

Maximiliano Mamone

Project Objective

Exploring classical (non-learning) fusion of the colour stereo cameras and the LiDAR of the KITTI 3D Object Detection benchmark at the three usual levels (early, mid and late), and comparing each of them with the single sensors on the same frames and with the same metrics.

Project Overview

The starting point is the LiDAR-only pipeline of the previous project (L1: RANSAC floor removal, log-polar BEV grid, clustering, oriented box fitting and rule based classification), reused as is. The camera side is the colour stereo pair (image_2 / image_3, rectified and ~0.54 m apart), which gives depth without any learning.

Everything is evaluated on the standard val split (3769 frames) and every parameter was chosen on train frames (3712), so the L1 numbers differ from the previous report only because of the frame set.

Camera only: stereo to pseudo-LiDAR (C1)

Disparity is computed with OpenCV's semi-global block matching on the grayscale pair (128 disparities, 7×7 blocks, P1 = $8 \cdot 7^2$, P2 = $32 \cdot 7^2$, uniqueness ratio 10, speckle filter and a 1 px left-right check). Since the pair is rectified, depth comes from similar triangles:

$$Z = f \cdot B / d$$

with the focal f and the baseline B read from P2 and P3. The error grows with the square of the distance ($\Delta Z \approx Z^2 / (f \cdot B) \cdot \Delta d$, about 1 m at 40 m for a quarter of a pixel), so I keep only pixels up to 40 m and drop the “flying pixels” smeared between foreground and background at object edges (depth jump above 5% in their 3×3 neighbourhood). Every remaining pixel is back-projected and moved to the LiDAR frame, so that the L1 pipeline can run on it; on frame 8 the median depth error against the LiDAR is 0.17 m (1.7%).

The stereo cloud has almost three times the points of the LiDAR one (~330k), all in front of the car and much noisier, so three settings change: only points between 0.3 and 2.5 m above the ground are kept, and a BEV cell is occupied only with at least 25 points (with 1 point, as for the LiDAR, the whole scene merged into a handful of clusters).

Early fusion (FE)

The raw points of the two sensors go into a single cloud and the L1 pipeline runs on it. Adding the whole stereo cloud breaks the LiDAR clusters (Car F1 on train from 0.25 to 0.04), so stereo points are added only beyond 30 m, where the LiDAR rings get sparse, downsampled to one point per 20 cm cube to match the LiDAR density.

Mid fusion (FM)

Each sensor first turns its points into evidence on the same log-polar BEV grid with a simple inverse sensor model, and the grids are summed in log-odds (independent sensors and a 0.5 prior, so Bayes becomes a sum):

$$L = \Sigma_s \log(p_s / (1 - p_s))$$

For the LiDAR every object point in a cell adds log(0.6 / 0.4), up to 3 points, so an isolated point is weak evidence. Positive stereo evidence merged neighbouring objects again, so the stereo only contradicts: a cell where it sees ground and no object gets log(0.4 / 0.6). Cells with $L > 0.3$ are occupied and clustered as in L1, with boxes fitted on the LiDAR points only.

Late fusion (FL-c)

L1 and C1 each produce their final detections and the two lists are associated with the Hungarian algorithm on the BEV distance between box centres (gates of 2 m for cars, 1 m for pedestrians and 1.5 m for cyclists). I tried keeping single-sensor detections with a reduced confidence, but the best choice on train was to keep only the LiDAR detections confirmed by the camera, with box and class from the LiDAR, whose geometry is far more accurate.

A learned contrast on LiDAR only (L2)

To see how much of the gap comes from the hand-written decision step, the same L1 clusters are given to a small PointNet (a shared per-point MLP $3 \to 64 \to 128 \to 256$ with max pooling, plus box size, range and point count) with two heads: the class (background, Car, Pedestrian, Cyclist) and a 7-value correction of the box. It was trained on ~100k labelled clusters of train (20 epochs, ~5 minutes on the laptop GPU) and the per-class confidence thresholds were chosen on 300 held-out train frames.

Evaluation

Same as the previous project (greedy matching by 3D IoU, thresholds 0.7 Car and 0.5 Pedestrian/Cyclist, Moderate subset, Van and Person_sitting ignored), plus BEV IoU and a detection confusion matrix, which shows almost no class swaps for any method: the errors are missed and invented objects.

Results

Method	Car TP	Car FP	Car P	Car R	Car F1	Ped F1	Cyc F1
L1 LiDAR	3375	11179	0.232	0.429	0.301	0.014	0.053
C1 stereo	1246	13719	0.083	0.158	0.109	0.003	0.012
FE early	3030	9066	0.250	0.385	0.303	0.015	0.058
FM mid	3444	11129	0.236	0.437	0.307	0.012	0.048
FL-c late	3099	4747	0.395	0.394	0.394	0.030	0.123
L2 PointNet	3841	1736	0.689	0.488	0.571	0.358	0.423

Run on the 3769 val frames, 3D IoU, Moderate difficulty.

The camera alone is far behind at these thresholds because of the stereo depth error. Early and mid fusion barely move L1 (+0.002 and +0.006 Car F1): the stereo noise is added to raw points or cells exactly where the LiDAR already works. Late fusion is clearly the best classical method (Car F1 from 0.301 to 0.394, cyclists more than doubled, 58% fewer car false positives): with a noisy second sensor, fusing decisions is more robust than fusing data. Still, L2 shows that the main bottleneck is the rule based classification: with the same LiDAR and the same clusters a learned decision almost doubles Car F1 and makes pedestrians and cyclists usable.

Limits: one operating point per method (no AP, since the classical methods have no confidence), parameters tuned on 150–300 train frames, and L2's recall capped by the L1 clustering it starts from.

AI Use Disclosure

As in the first project, I used an AI assistant above all as a tutor, to understand in depth the mathematics behind the methods (stereo geometry and its error, Bayesian fusion in log-odds, Hungarian association) and the PointNet architecture. The code is a mix of components carried over from the first project and additional parts developed by myself with AI assistance, mainly for debugging and performance-improvement purposes.